FULL STACK ENGINEER
PRODUCT DEVELOPMENT ASSESSMENT

Project: Multi-Tenant Customer Support Platform | **Format:** Take-home build + architecture review

Suggested effort: 2-3 days (time-box the implementation; document unfinished items). **Review:** 60-90 minutes.

1. Business scenario

A SaaS company wants to offer a customer support product to multiple organizations. Each organization has its own staff and customers. Customers submit support tickets; agents respond and update them; administrators oversee workloads and service quality. The product must protect each organization's data from access by other organizations.

You are the engineer responsible for taking this concept from requirements through a working MVP. Make reasonable assumptions where information is missing and document them. We are interested in your engineering decisions as much as the final implementation.

2. User roles and workflows

- **Organization administrator:** manage organization membership and roles; view all organization tickets and dashboard.
- **Support agent:** view assigned or permitted tickets, respond, change status and priority, and assign tickets where authorized.
- **Customer:** create tickets and view or comment on their own tickets.

Define the precise permissions and any additional business rules you believe are necessary.

3. Core functional requirements

- Authentication and authorization: secure sign-in and role-based access controls.
- Organization isolation: users must not read or modify another organization's data.
- Ticket lifecycle: create, view, update, assign, prioritize and close tickets. Define valid status transitions.
- Ticket discovery: paginated listing, search, filters (status, priority, assignee, customer) and sorting.
- Ticket conversation and audit history: comments plus a record of meaningful changes.
- Dashboard: counts by status/priority, overdue tickets and agent workload. State how metrics are calculated.
- Notifications: design a way to notify users about assignments or important changes; implementation may be scoped as a stretch goal.

4. Frontend expectations

- Build a usable responsive React application with login, ticket list, ticket detail and a simple dashboard.
- Show loading, empty, error and permission-denied states; validate user input.
- Explain component boundaries, API integration and state-management choices.
- Consider accessibility, optimistic versus confirmed updates, and concurrent edits.

5. Backend and API expectations

- Implement a Node.js API using Express or NestJS (or justify another choice).
- Provide documented endpoints for authentication, tickets, comments, assignments and dashboard data.
- Enforce authorization and tenant isolation on the server, not only in the UI.
- Use validation, consistent error responses, structured logging and sensible HTTP status codes.
- Explain how you would handle duplicate requests, concurrent updates and failed external integrations.

6. Database and data access

- Choose PostgreSQL or MongoDB; explain why it suits the access patterns.
- Design entities/collections for organizations, users, memberships/roles, tickets, comments and activity events.
- Show keys/relationships or document structure, constraints, tenant-scoped queries and indexes.
- Demonstrate pagination and filtering. Explain how the design behaves with millions of tickets.
- Include seed data for at least two organizations and show how isolation is verified.

7. Cloud and operational design

- Draw a deployment architecture using AWS or a cloud provider of your choice.
- Discuss application hosting, managed database, secrets, TLS, networking, storage and backup/restore.
- Describe CI/CD, environments, migrations, monitoring, alerts, rollback and approximate cost drivers.
- Explain what you would change as usage grows from hundreds to tens of thousands of users.

8. Implementation scope and deliverables

Required working vertical slice: login; organization-aware authorization; ticket creation, listing, filtering and detail/update; persistent database; minimal dashboard; basic automated tests; and local Docker setup. Prioritize correctness and security over breadth.

Submit: (1) source repository, (2) README with setup and run instructions, (3) architecture diagram, (4) data model and key indexes, (5) API documentation/examples, (6) sample credentials or seed instructions, (7) test instructions/results, and (8) a short tradeoffs/limitations/next-steps note.

Optional: deployment URL, CI pipeline, notifications, file attachments, richer analytics, and observability dashboard. Clearly identify optional work.

9. Review discussion

Be ready to walk through your user journeys, demonstrate the application, explain design choices and discuss how you would investigate slow queries, prevent cross-tenant access, handle simultaneous ticket edits, recover from an outage, and roll back a faulty release. There is no required microservices architecture or mandatory cloud service list; explain the simplest solution that meets the requirements.

10. Submission guidance

Use your preferred development tools, libraries and AI assistants if helpful, but be prepared to explain and modify every important part of the solution. State what you implemented personally, what you reused, and any generated

code you relied on. Record assumptions and known gaps rather than silently omitting requirements.

Success criteria: A coherent end-to-end product slice, sound technical reasoning, secure data boundaries, clear communication and awareness of production tradeoffs.